A simple technique for separating keywords from identifiers is to initialize appropriately the symbol table in which information about identifiers is saved. For the tokens of Fig. 3.10 we need to enter the strings `if`, `then`, and `else` into the symbol table before any characters in the input are seen. We also make a note in the symbol table of the token to be returned when one of these strings is recognized. The return statement next to the accepting state in Fig. 3.13 uses *gettoken*() and *install_id*() to obtain the token and attribute-value, respectively, to be returned. The procedure *install_id*() has access to the buffer, where the identifier lexeme has been located. The symbol table is examined and if the lexeme is found there marked as a keyword, *install_id*() returns 0. If the lexeme is found and is a program variable, *install_id*() returns a pointer to the symbol table entry. If the lexeme is not found in the symbol table, it is installed as a variable and a pointer to the newly created entry is returned.

The procedure *gettoken*() similarly looks for the lexeme in the symbol table. If the lexeme is a keyword, the corresponding token is returned; otherwise, the token **id** is returned.

Note that the transition diagram does not change if additional keywords are to be recognized; we simply initialize the symbol table with the strings and tokens of the additional keywords. □

The technique of placing keywords in the symbol table is almost essential if the lexical analyzer is coded by hand. Without doing so, the number of states in a lexical analyzer for a typical programming language is several hundred, while using the trick, fewer than a hundred states will probably suffice.

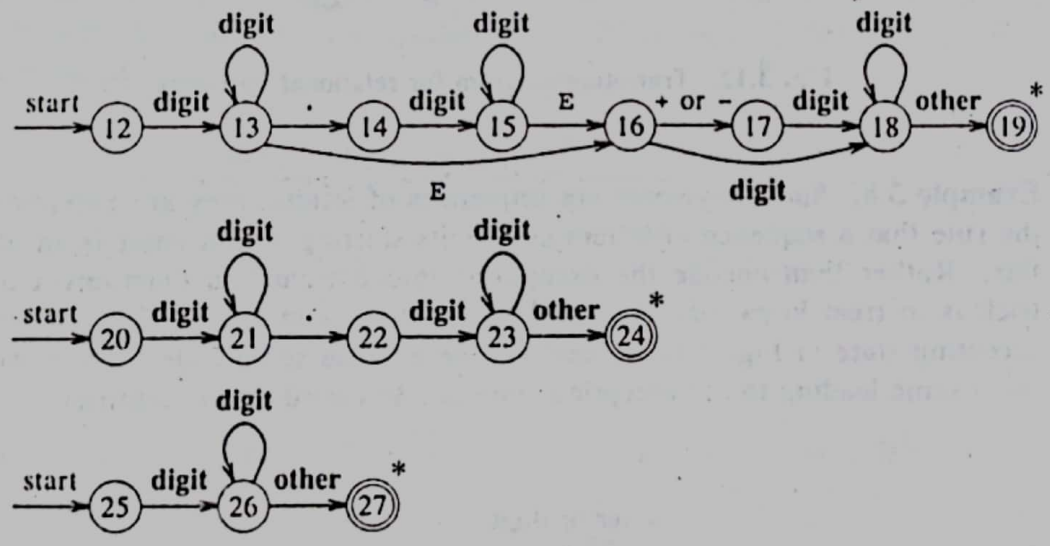


Fig. 3.14. Transition diagrams for unsigned numbers in Pascal.

Example 3.9. A number of issues arise when we construct a recognizer for unsigned numbers given by the regular definition

num → **digit**$^+$ (.**digit**$^+$)? (E(+|-)? **digit**$^+$)?

Note that the definition is of the form **digits fraction? exponent?** in which **fraction** and **exponent** are optional.

The lexeme for a given token must be the longest possible. For example, the lexical analyzer must not stop after seeing 12 or even 12.3 when the input is 12.3E4. Starting at states 25, 20, and 12 in Fig. 3.14, accepting states will be reached after 12, 12.3, and 12.3E4 are seen, respectively, provided 12.3E4 is followed by a non-digit in the input. The transition diagrams with start states 25, 20, and 12 are for **digits**, **digits fraction**, and **digits fraction? exponent**, respectively, so the start states must be tried in the reverse order 12, 20, 25.

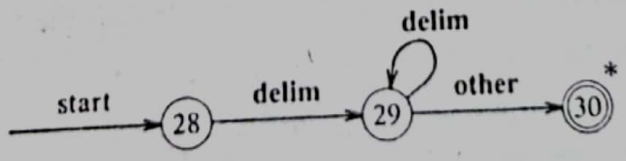
The action when any of the accepting states 19, 24, or 27 is reached is to call a procedure *install_num* that enters the lexeme into a table of numbers and returns a pointer to the created entry. The lexical analyzer returns the token **num** with this pointer as the lexical value. □

Information about the language that is not in the regular definitions of the tokens can be used to pinpoint errors in the input. For example, on input 1.<x, we fail in states 14 and 22 in Fig. 3.14 with next input character <. Rather than returning the number 1, we may wish to report an error and continue as if the input were 1.0<x. Such knowledge can also be used to simplify the transition diagrams, because error-handling may be used to recover from some situations that would otherwise lead to failure.

There are several ways in which the redundant matching in the transition diagrams of Fig. 3.14 can be avoided. One approach is to rewrite the transition diagrams by combining them into one, a nontrivial task in general. Another is to change the response to failure during the process of following a diagram. An approach explored later in this chapter allows us to pass through several accepting states; we revert back to the last accepting state that we passed through when failure occurs.

Example 3.10. A sequence of transition diagrams for all tokens of Example 3.6 is obtained if we put together the transition diagrams of Fig. 3.12, 3.13, and 3.14. Lower-numbered start states are to be attempted before higher numbered states.

The only remaining issue concerns white space. The treatment of ws, representing white space, is different from that of the patterns discussed above because nothing is returned to the parser when white space is found in the input. A transition diagram recognizing ws by itself is



Nothing is returned when the accepting state is reached; we merely go back to the start state of the first transition diagram to look for another pattern.